

MemoryAgentBench — FactConsolidation, September 2026

Kaleidoscope measured against BM25 over the same units, a no-memory control, and a full-context control that puts the entire corpus in the prompt.

Every arm uses the **same reader** (gpt-5.6-luna, reasoning effort high, a 10-token visible answer cap) and the same corpus. The only variable is what reaches the reader’s prompt. Every `kscope` call shells out to the installed binary, verified with `kscope model` reporting `"status": "bundled"`.

The question

Not “is retrieval accurate”. The question a harness actually faces:

Can a memory store cut the tokens an agent spends on a task while raising the rate at which it solves it — by supplying the relevant prior knowledge instead of the whole history?

Accuracy alone rewards stuffing the context window. Tokens alone rewards serving nothing. **Both appear in every cell below, and neither is quoted without the other.**

Headline

At the longest context the benchmark offers, retrieval **beats** whole-corpus prompt-stuffing on accuracy while spending **0.4% of the tokens**.

262k, single-hop	accuracy (SEM/100)	reader tokens/question
full context (entire corpus in prompt)	85 *	291,746
kscope	90	1,195
BM25 over the same units	78	2,066
no memory	13	189

* Complete at 100 of 100 questions, but scored by `score_partial.py` rather than by the benchmark’s own scorer, because the run process stopped before the harness’s end-of-run scoring. That script’s one assumption is stated in its header. Treat this cell as provisional until the harness reproduces it.

This number moved as the run progressed — 90% at 60 questions, 88% at 76, 85% at 100. The early reads were the most favourable this cell had, which is the reason not to quote a running cell as settled.

Scores

800 questions. `substring_exact_match`, single-hop / multi-hop, out of 100 per cell:

context length	no memory	BM25 units	kscope
6k	19 / 0	97 / 33	95 / 15
32k	26 / 0	91 / 13	95 / 8
64k	24 / 0	85 / 13	96 / 10
262k	13 / 2	78 / 11	90 / 10

Single-hop is the length story. BM25 decays from 97 to 78 as the corpus grows; kscope holds 95 to 90. The gap opens as the haystack grows, which is the only regime where a memory store is interesting at all.

Candidate recall@10 — whether the unit holding the answer was among the ten the read path considered:

context length	kscope	BM25 units
6k	100%	96.9%
32k	100%	93.8%

context length	kscope	BM25 units
64k	100%	86.9%
262k	98%	83.8%

And it costs less. At 262k, kscope serves a mean of 5.07 units for 1,195 reader tokens; BM25 serves a fixed 10 for 2,066. **Twelve accuracy points more, on 42% fewer tokens.**

Where Kaleidoscope loses

Multi-hop, at every length. BM25 beats kscope 33 to 15 at 6k, and stays ahead at 32k, 64k and 262k. This is a real loss and it is not a sampling artefact.

A probe suite of 5,258 retrievals with **zero model calls** was run to find the cause. It ruled out the explanation that had been assumed:

- The second hop’s unit is usually **already retrievable** — at 6k it is always in the candidate set, and reach only starts to bind at 64k. The failure is in ranking and cutting, not in coverage.
- A second retrieval round keyed on the bridge entity closes a substantial share of chains **with no graph traversal involved**.
- Under the same oracle conditions, **BM25 closes more chains than kscope does**, including on kscope’s own seed strings.

An earlier version of this analysis reported a much higher closure rate. That probe took its bridge entity from the gold answer chain, which is information no retriever has at query time. The figure was an oracle and is withdrawn; the honest oracle-free rate is roughly 55% of chains.

Two ceilings the dataset imposes

Gold sometimes contradicts the benchmark’s own rule. FactConsolidation states that the newest serial for a fact wins, but scores some questions against an answer only reachable through an older serial. That affects 2-3% of single-hop and **13-34% of multi-hop** questions. No arm can exceed that ceiling, and multi-hop numbers should be read against it rather than against 100.

The full-context control is not an upper bound. Its errors are confident wrong entities, consistent with a reader shown both the current and the superseded value picking the superseded one. Retrieval filters a distractor that stuffing surfaces. That is a hypothesis consistent with the errors, not a measured claim.

Methodology notes worth copying

Pin the binary by hash, and re-check it per call. During this campaign the installed binary was replaced by an unrelated process while a run was in flight. Twenty calls executed against the wrong build before it was noticed, and that run was discarded rather than repaired.

The harness *had* a hash check and it did not fire, because the check memoised its result in a module-level flag: a long-lived worker verified once at startup and never again, while every call re-resolved the path. **A one-shot per-process check cannot see a mid-run swap.** Each run now resolves its own private copy of the binary and names the resolved path and hash in its report.

Providers refuse prompts you did not expect. Reading mathematical notation from reinforcement-learning papers triggered 400 Invalid prompt: your prompt was flagged as potentially violating our usage policy on a meaningful fraction of calls, and at different rates in different arms. Rather than assume it washed out, degraded questions were dropped **pairwise** from both arms: the estimate moved from +0.0177 to +0.0178. It does not confound the comparison, and that is now demonstrated rather than asserted.

Score the arms you are comparing on the same instrument. Reader model, reasoning effort and answer cap are identical in every cell here. A reader difference is indistinguishable from a memory difference in the final number.

Reproducing each number

Every number above comes from one of these commands, run from `contrib/bench-2026-09-20/` after replacing the `/path/to/kaleidoscope` placeholders (see that directory's README).

number	command
the 4 x 3 score matrix, single- and multi-hop	<code>. mab/env.sh && . mab/env_full.sh && python mab/drive_full.py --run_id <id> --lengths 6k,32k,64k,262k</code>
the generated score tables and candidate recall@10	<code>python mab/full_check.py <id> --lengths 6k,32k,64k,262k --out reports/<id>.md</code>
the full-context run	<code>. mab/env.sh && . mab/env_fullctx.sh && python mab/drive_fullctx.py --run_id <id></code>
the full-context 85, and its token count	<code>python mab/score_partial.py <MemoryAgentBench parquet> <that run's adapter log></code>

The harness predates this repository's four-phase `kbench` shape and does not yet conform to it; the directory README says what would need to change.

Raw per-question rows are deliberately not committed. A finding belongs in the write-up with the command that produced it; the rows are evidence for the write-up rather than an artefact to read from git.